# Day Final - Team NTU

OpenACC
More Science, Less Programming

# Team NTU - Members

Organization of all team members: National Taiwan University

Yu-Chi Cheng



Hao-Kai Chang



Yu-Jen Liu



Prof.Chun-Yi Lee



Yun-Yang Liao



Fan-Shi Liu



Yu-Chen Sun



OpenACC More Science, Less Programming

OPEN HACKATHONS

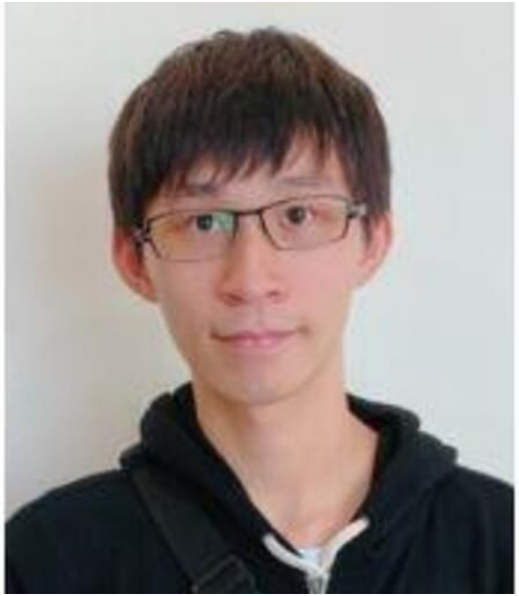
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing

NVIDIA

# Team NTU - Mentors



Rosie Wu - NVidia

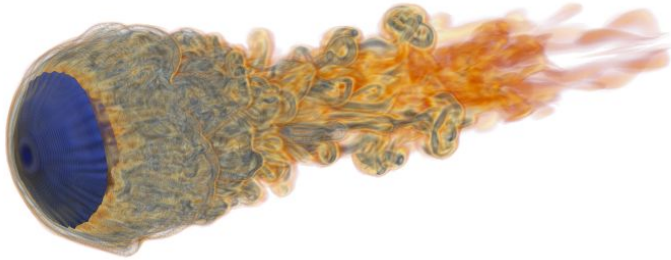


Johnson Sun - NVidia

# Multicomponent Flow Code (MFC)

## An open-source exascale flow solver, 2025 Gordon Bell Prize finalist

- **Wilfong et al. (2025), MFC 5.0:** open-source solver for compressible, multiphase flow, used in aerospace, medicine and energy
- **2025 Gordon Bell Prize finalist:** 200 trillion grid points on El Capitan and Frontier, the largest public CFD simulation to date
- Runs on GPUs through Fortran + OpenACC
- **We build on it** to speed up its WENO + HLLC solver on NCHC's H200 GPUs



A shock wave breaking up a droplet in 3D

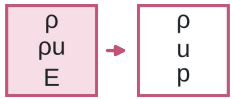
Source: Wilfong et al., "MFC 5.0: An exascale many-physics flow solver", arXiv:2503.07953 (2025); Georgia Tech Research news, 2025 Gordon Bell Prize.

# MFC Algorithm

## One Runge-Kutta stage: every cell is updated from its neighbours

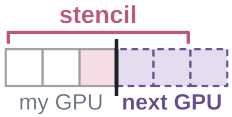
repeated for each direction x, y, z

**Conservative → primitive**




Stored values → density, velocity, pressure

**Ghost cells**



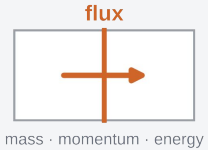

Cells at the edge need values from the next GPU

**WENO reconstruction**



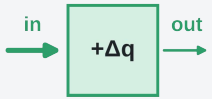

Estimate the values on each cell face

**HLLC Riemann solver**



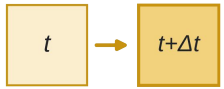

Mass, momentum and energy crossing each face

**Flux difference**




Flow in minus flow out = change of each cell

**RK update**




Move every cell one sub-step forward

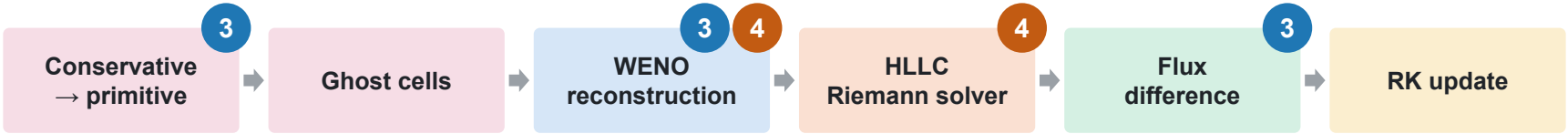
**Grind time: the time of this whole stage**

Our speed metric, normalised per cell and per equation. lower is better. 3 stages = 1 time step.

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

# Optimization Strategy

## Guided by profiling: first remove GPU idle time, then make the kernels faster




1 Launch overhead: the GPU sat idle between every pair of kernels

2 Overlap: halo exchange and the x, y, z sweeps run at the same time

### 1 Launch overhead

Asynchronous queue, CUDA Graph, fewer GPU syncs

**Grind time −34%**

### 2 Comm. overlap

Parallel x/y/z sweeps, non-blocking halo exchange

**Grind time −17%**

### 3 Memory traffic

Access order, fewer copies, kernel fusion

**Grind time −40%**

### 4 Compiler

Full inlining of HLLC, inline budget, register caps

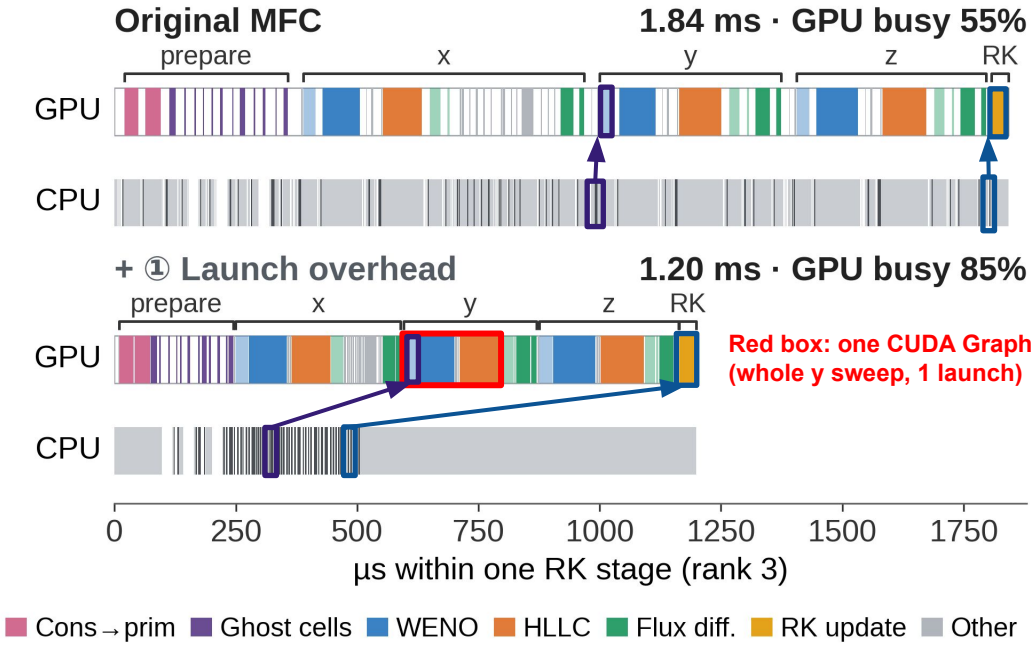
**Grind time −19%**

**All four are bit-identical** with original MFC. Changes that alter round-off come after, as a separate setting

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

# ① Launch Overhead

## Closing the gaps between kernels keeps the GPU busy




**−34%**

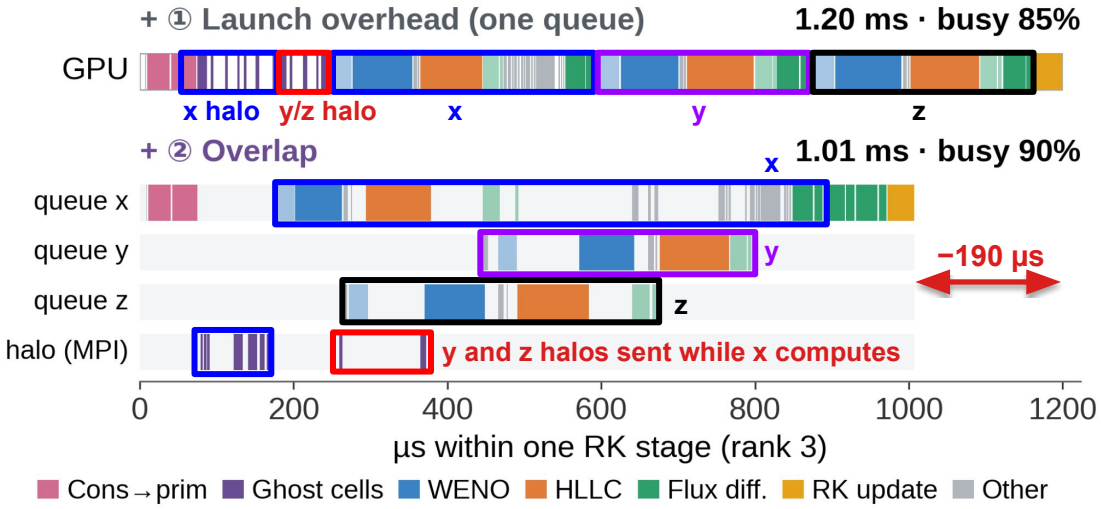
Grind time 0.112 → 0.074 ns, 8 x H200

- **Asynchronous OpenACC queue:** the CPU queues every kernel and waits only before MPI or host reads
- **Fewer syncs:** index ranges uploaded once, not before every sweep
- **CUDA Graph:** one graph per direction, launched as soon as its halo arrives, plus one for the RHS sum; CPU launches per stage 44 → 13

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

# ② Concurrency & Communication Overlap

## x, y and z sweeps run side by side, and halos travel while the GPU computes




**−17%**

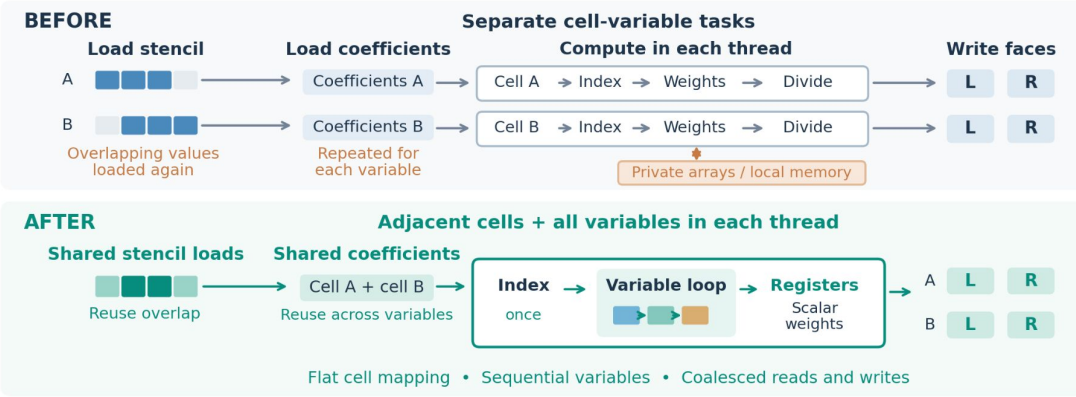
Grind time 0.074 → 0.062 ns, 8 x H200

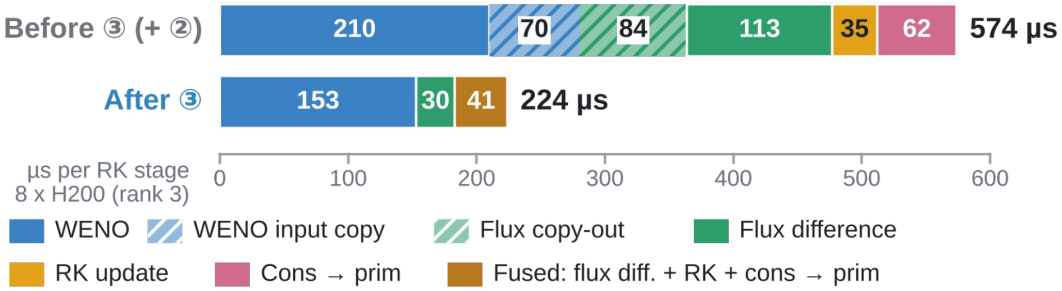
- **Parallel directions:** x, y, z sweeps on three queues; each rebuilds only the rows it needs
- **y/z halo overlap:** halos exchanged while x computes
- **Non-blocking halo:** all halos sent at once

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

# ③ Reducing Memory Traffic

## These kernels wait on memory, so we touch DRAM less often and more efficiently




**−40%**

Grind time 0.062 → 0.037 ns, 8 x H200

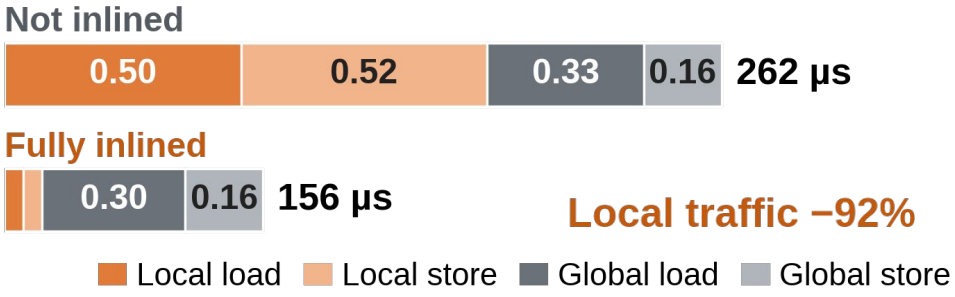
- **Data reuse:** Each thread handles adjacent cells and all variables, reusing inputs and keeping intermediate values in registers.
- **Fewer copies:** flux written in place, WENO input copy skipped
- **Kernel fusion:** cons → prim with the state copy (286 → 83 µs), flux diff + RK update + next cons → prim

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

# ④ Compiler: Inlining & Registers

**Same source code, different build: the compiler now sees the whole HLLC solver**

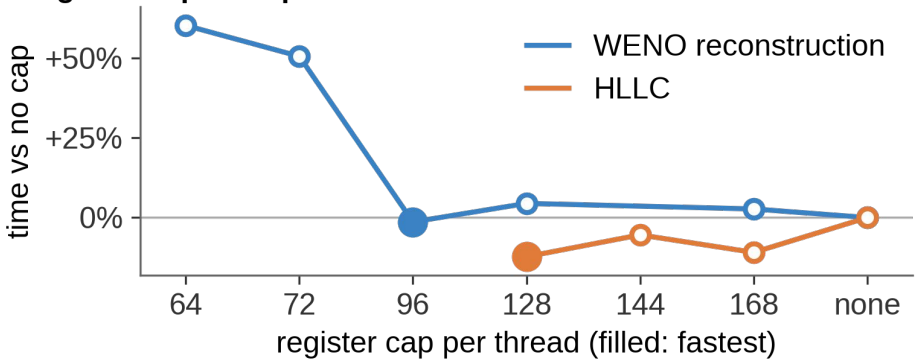
**Full inlining of HLLC: memory traffic per RK stage (GB)**




**−19%**

Grind time 0.037 → 0.030 ns, 8 x H200

**Register cap sweep**




- **Full inlining:** HLLC's 8 device calls built in one compile unit; local-memory traffic −93%, HLLC −30%
- **Register caps:** a larger inline budget and a register cap per kernel, so more threads fit on each SM

# Beyond Bit-Identical

## Changes that reorder floating-point math. off by default

- **HLLC 5-equation rewrite:** HLLC specialized for the 5-equation model; limiter values stay in registers, branches become a switch
- **WENO common denominator:** one reciprocal per cell face instead of several divisions

**−20%**

Grind time 0.030 → 0.024 ns, 8 x H200

**MFC** — 4 divisions

$$\alpha_q = \frac{d_q}{\beta_q} \qquad \omega_q = \frac{\alpha_q}{\alpha_0 + \alpha_1}$$

multiply by $\beta_0\beta_1$ →

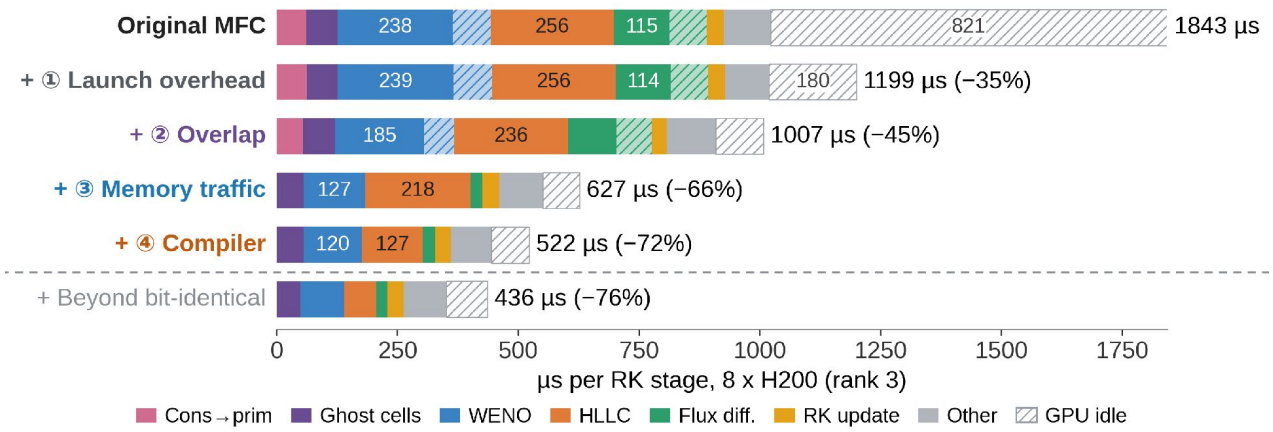
**Ours** — 1 reciprocal

$$\omega_0 = \frac{d_0\beta_1}{d_0\beta_1 + d_1\beta_0} \qquad \omega_1 = \frac{d_1\beta_0}{d_0\beta_1 + d_1\beta_0}$$

**Not bit-identical, but they still pass 62 representative MFC test cases**

# Final Performance

## Grind time fell 76%, from 0.112 to 0.024 ns




**17.6x faster**

**Ours on 8 x H200 vs. original MFC on 1 x H200 (14.1x bit-identical 4.6x on the same 8 GPUs)**

| Measure | Original MFC | Ours, bit-identical | Ours, fastest |
|---|---|---|---|
| **8 x H200, grind time** | 0.1117 ns | 0.0303 ns(3.69x) | 0.0243 ns(4.60x) |
| **1 x H200, grind time** | 0.4267 ns | 0.1561 ns (2.73x) | 0.1114 ns (3.83x) |
| **8 x H200, whole run** | 280.6 s | 87.5 s (3.21x) | 71.3 s (3.93x) |

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

# Energy Efficiency

| INPUTS | | |
|---|---|---|
| Baseline | GPU | *Select the baseline as GPU or CPU* |
| Baseline GPU Type | 8x H200 141GB SXM5 | *Select the CPU/GPU Type or GPU Node configuration* |
| Baseline GPU # GPUs | 1 | *Select the number of CPU cores or GPUs (within one node)* |
| Final GPU Node | 8x H200 141GB SXM5 | *Select the final GPU Type or GPU Node configuration* |
| Final # GPUs | 8 | *Select the number of GPUs (within one node)* |
| Application Speedup | 17.6x | *Enter the amount of achieved speedup* |

| | |
|---|---|
| **Node Replacement** | 2.2x |

| GPU NODE POWER SAVINGS | | | |
|---|---|---|---|
| | **Baseline** | **Comparison** | |
| | 8x H200 141GB SXM5 | 8x H200 141GB SXM5 | **Power Savings** |
| Compute Power (W) | 22,591 | 10,298 | 12,293 |
| Networking Power (W) | 1,931 | 880 | 1,051 |
| **Total Power (W)** | **24,522** | **11,178** | 13,344 |

| | |
|---|---|
| **Node Power efficiency** | **2.2x** |

| ANNUAL ENERGY SAVINGS PER GPU NODE | | | |
|---|---|---|---|
| | 8x H200 141GB SXM5 | 8x H200 141GB SXM5 | **Power Savings** |
| Compute Power (kWh/year) | 197,899 | 90,210 | 107,689 |
| Networking Power (kWh/year) | 16,911 | 7,709 | 9,202 |
| **Total Power (kWh/year)** | **214,810** | **97,919** | **116,891** |

| | | |
|---|---|---|
| **$/kWh** | $ | 0.18 |
| **Annual Cost Savings** | $ | 21,040.41 |
| **3-year Cost Savings** | $ | 63,121.22 |

| | |
|---|---|
| **Metric Tons of CO2** | 83 |
| **Gasoline Cars Driven for 1 year** | 18 |
| **Seedlings Trees grown for 10 years** | 1,370 |

(source: Link)

18

Gasoline cars driven for a year






1,370

Trees growing for 10 years

83

Metric tons of CO2



# How AI Agent Helped

## Unit tests first, then profile → change → verify for every optimization

### Unit test

- Tests written first
- Original MFC output as the reference

### Profile

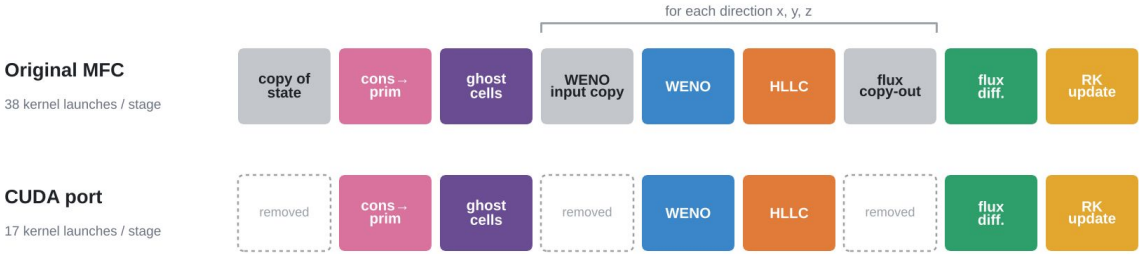
- Read Nsight Systems and Compute reports
- Draw profile figures

### Change

- Loop-order, layout and fusion rewrites
- One stage in CUDA

### Verify

- Unit tests + 62 cases
- md5 vs. original MFC




### CUDA port as a reference

A plain CUDA port, with no tuning, already runs at 0.114 ns on 1 x H200, about as fast as our fully optimized OpenACC (0.111 ns).

**Main path: Fortran + OpenACC**

**With the agent, the CUDA port took about 1 day instead of an estimated 3~4 weeks by hand**

# Final Thoughts & Wishlist

## Final Thoughts

- **Was it worth it?** Absolutely. We learned a lot, and every mentor meeting showed us something we had not thought of; that hands-on experience is the most valuable part
- **Will we continue?** Yes: open a pull request to upstream MFC with the OpenACC changes
- **Next steps:** run on multiple nodes and larger grids
- **Support we need:** more H200 node hours for multi-node runs

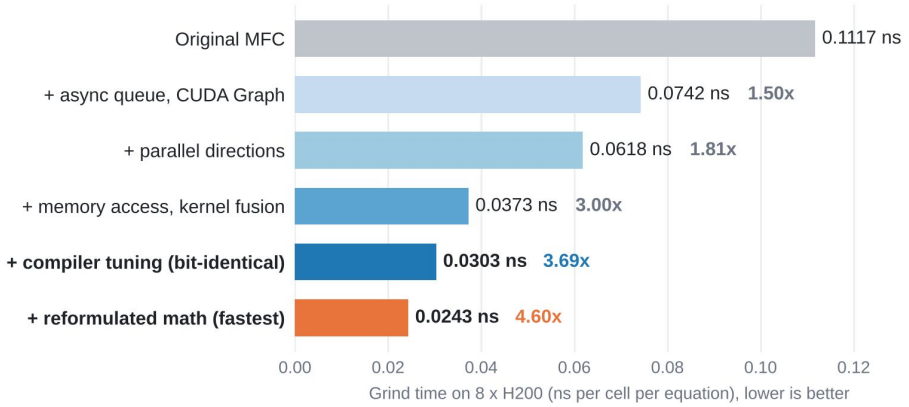
## Wishlist

- **Tools:** a per-loop register, spill and occupancy report for OpenACC
- **Event:** the weekly scrums and mentor meetings worked well for us :D
- **Systems:** more stable VS Code Remote-SSH connections to Nano4

## Application Background

MFC is an open-source solver for compressible multiphase flow, such as shock waves breaking up droplets and collapsing bubbles. Finer grids show more physics but cost about 16x per halving of the cell size in 3D.

Motif: structured-grid stencil (WENO reconstruction, HLLC Riemann solver, 3-stage Runge-Kutta).




## Hackathon Objectives and Approach

- Fortran + OpenACC on 8 x H200 (Nano4)
- Nsight Systems and Nsight Compute to find hot spots
- Async queues and parallel x/y/z sweeps to remove GPU idle time
- HLLC inlining and rewrite; WENO loop order and data layout
- CUDA port of one RK stage as an upper bound
- Claude Code for profiling, porting and tests

## Technical Accomplishments and Impact

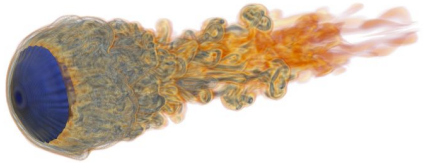
- Against original MFC on the same GPUs: 3.69x on 8 x H200 (grind time 0.1117 → 0.0303 ns), 2.73x on 1 x H200
- Results bit-identical, or (fastest setting) pass 62 representative MFC test cases
- Energy: ~2.2x less GPU energy per simulation
- The same allocation runs more cases or finer grids

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

# A storyline for publication on NCHC's website

震波擊碎液滴的多相流模擬

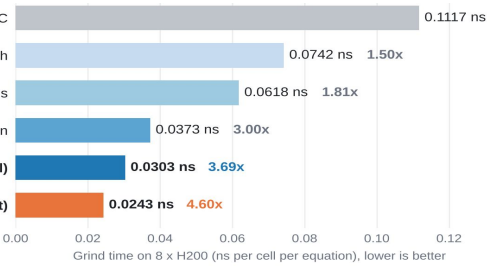
研究領域示意圖



**台大 Team NTU 團隊來自 李濬屹 老師帶領的 ElsaLab實驗室，**
**AI Agent 將優化工作流程加速了 30 倍；**
**GPU 將 可壓縮多相流模擬程式 MFC 研究加速了 3.7 倍！！**

飛機以超音速穿過雨區時，雨滴會被震波打碎並侵蝕機身；醫療上用震波擊碎腎結石時，周圍的氣泡也會瞬間塌縮、產生強烈衝擊。這些現象發生得極快、尺度極小，難以用實驗直接觀察，只能靠電腦模擬來預測。
但這類模擬非常耗時：要看清液滴碎裂的細節，網格必須很細，而網格每加密一倍，計算量就增加約 16 倍。
我們在國網中心的 NVIDIA H200 上分析開源模擬程式 MFC，減少 GPU 空等與資料搬移，並讓三個方向同時計算；過程中也用 AI 工具 Claude Code 協助分析與測試。在結果與原版完全相同的前提下，8 張 H200 的計算速度提升 3.7 倍，整個模擬從 280 秒縮短到 88 秒，估計耗電只剩約四分之一。
更快的模擬讓研究者能用同樣的時間與電力跑更細的網格、試更多設計，加速飛行器防護與醫療碎石等應用的研發。

實驗結果




THANK YOU!!!